

Punto 5.4 — Transacciones: SaveChanges y transacciones explícitas

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.3. La carpeta M05/PROYECTO/5.4 contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Demostrar atomicidad de `SaveChanges`, `Commit`, `Rollback` y `savepoints` reales con SQL Server y MARS desactivado.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.4
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/TransaccionesM5Repositorio.cs`
- `src/AceriaData.Console/appsettings.json`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar TransaccionesM5Repositorio.cs

```
public ResultadoTransaccionM5Dto DemostrarAtomicidadSaveChanges()
{
    const string numeroValido = "0F-M5-54-ATOMIC";
    Limpiar(numeroValido);
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

    context.OrdenesFabricacion.Add(CrearOrden(numeroValido));
    context.OrdenesFabricacion.Add(CrearOrden(NumeroExistente));

    var falloEsperado = false;
    try
    {
        context.SaveChanges();
    }
    catch (DbUpdateException)
    {
        falloEsperado = true;
        context.ChangeTracker.Clear();
    }

    var existeValida = Existe(numeroValido);
    return new ResultadoTransaccionM5Dto(
        "Atomicidad de un único SaveChanges",
        falloEsperado && !existeValida,
        existeValida,
        Existe(NumeroExistente),
        MarsHabilitado(context),
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — TransaccionesM5Repositorio.cs

Línea 1: `public ResultadoTransaccionM5Dto DemostrarAtomicidadSaveChanges()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `const string numeroValido = "0F-M5-54-ATOMIC";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 4: `Limpiar(numeroValido);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 5: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `context.OrdenesFabricacion.Add(CrearOrden(numeroValido));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 9: `context.OrdenesFabricacion.Add(CrearOrden(NumeroExistente));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 11: `var falloEsperado = false;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 13: `{` → Delimita el bloque sintáctico correspondiente.

Línea 14: `context.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 15: `}` → Delimita el bloque sintáctico correspondiente.

Línea 16: `catch (DbUpdateException)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 17: `{` → Delimita el bloque sintáctico correspondiente.

Línea 18: `falloEsperado = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `context.ChangeTracker.Clear();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 20: `}` → Delimita el bloque sintáctico correspondiente.

Línea 22: `var existeValida = Existe(numeroValido);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 23: `return new ResultadoTransaccionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 24: `"Atomicidad de un único SaveChanges"`, → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 25: `falloEsperado && !existeValida,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 26: `existeValida,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 27: `Existe(NumeroExistente),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 28: `MarshHabilitado(context),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 29: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 30: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar TransaccionesM5Repositorio.cs

```
public ResultadoTransaccionM5Dto DemostrarRollbackASavepoint()
{
    const string numero1 = "0F-M5-54-SP-1";
    const string numero2 = "0F-M5-54-SP-2";
    const string savepoint = "AntesSegundaOrden";
    Limpiar(numero1, numero2);

    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var mars = MarsHabilitado(context);
    if (mars)
        throw new InvalidOperationException("5.4 requiere MultipleActiveResultSets=false.");

    using (var transaction = context.Database.BeginTransaction())
    {
        context.OrdenesFabricacion.Add(CrearOrden(numero1));
        context.SaveChanges();
        transaction.CreateSavepoint(savepoint);

        context.OrdenesFabricacion.Add(CrearOrden(numero2));
        context.SaveChanges();
        transaction.RollbackToSavepoint(savepoint);

        context.ChangeTracker.Clear();
        transaction.Commit();
    }

    return new ResultadoTransaccionM5Dto(
        "Rollback a savepoint",
        Existe(numero1) && !Existe(numero2),
        Existe(numero1), Existe(numero2), mars,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — TransaccionesM5Repositorio.cs

Línea 1: `public ResultadoTransaccionM5Dto DemostrarRollbackASavepoint()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `const string numero1 = "0F-M5-54-SP-1";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 4: `const string numero2 = "0F-M5-54-SP-2";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 5: `const string savepoint = "AntesSegundaOrden";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 6: `Limpiar(numero1, numero2);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 8: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 9: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 10: `var mars = MarsHabilitado(context);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `if (mars)` → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.

Línea 12: `throw new InvalidOperationException("5.4 requiere MultipleActiveResultSets=false.");` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 14: `using (var transaction = context.Database.BeginTransaction())` → Abre una transacción explícita sobre la conexión del DbContext.

Línea 15: `{` → Delimita el bloque sintáctico correspondiente.

Línea 16: `context.OrdenesFabricacion.Add(CrearOrden(numero1));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 17: `context.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 18: `transaction.CreateSavepoint(savepoint);` → Crea un punto intermedio al que puede volver la transacción sin descartar todo el trabajo previo.

Línea 20: `context.OrdenesFabricacion.Add(CrearOrden(numero2));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 21: `context.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 22: `transaction.RollbackToSavepoint(savepoint);` → Revierte solo las operaciones posteriores al savepoint indicado.

Línea 24: `context.ChangeTracker.Clear();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 25: `transaction.Commit();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 26: `}` → Delimita el bloque sintáctico correspondiente.

Línea 28: `return new ResultadoTransaccionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 29: `"Rollback a savepoint",` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 30: `Existe(numero1) && !Existe(numero2),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 31: `Existe(numero1), Existe(numero2), mars,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 32: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 33: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar appsettings.json

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\
\\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResult
Sets=false;"
  }
}
```

Explicación línea a línea — appsettings.json

Línea 1: `{` → Delimita el bloque sintáctico correspondiente.

Línea 2: `"ConnectionStrings": {` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 3: "AceriaDB": "Server=(localdb)\

\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResultSets

→ Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 4: } → Delimita el bloque sintáctico correspondiente.

Línea 5: } → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-
project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con 5.4 OK. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Un único SaveChanges revierte todo el conjunto si una escritura falla.
- Una transacción explícita puede agrupar varias llamadas a SaveChanges.
- Rollback deshace una escritura ya enviada antes del fallo posterior.
- RollbackToSavepoint conserva lo anterior al savepoint y revierte lo posterior.
- MultipleActiveResultSets=false en el laboratorio de savepoints.

Reto resuelto y ampliación

Reto: Introduce un tercer SaveChanges después del savepoint y decide qué parte debe persistir tras un rollback parcial.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Dar por hecho que Commit fallido siempre implica un estado conocido.
- Usar MARS y esperar savepoints automáticos de EF Core.
- Presentar RELEASE SAVEPOINT como operación necesaria de SQL Server.
- Mantener transacciones abiertas durante trabajo ajeno a la base.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador 5.4 OK.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.4, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.4 OK

Conexión con el siguiente punto

El siguiente estado es 5.5 y parte directamente de este proyecto completo.
